

Funções e Métodos em C#

Do primeiro método aos Arrays e Listas

Manual do Formando

Contexto prático: Sistema de Gestão de Alunos

Índice

1. Introdução	3
2. Anatomia de um método	4
2.1 Onde vivem os métodos: a classe e o Main	5
3. Tipos de métodos	7
3.1 Métodos void (sem retorno)	8
3.2 Métodos com retorno	11
3.3 Parâmetros por valor	14
3.4 Parâmetro ref	16
3.5 Parâmetro out	18
3.6 Parâmetro params	19
3.7 Parâmetros opcionais	20
3.8 Sobrecarga (overloading)	21
4. Arrays	22
5. Listas (List<T>)	24

1. Introdução

Quando escrevemos programas, rapidamente percebemos que estamos sempre a repetir as mesmas tarefas: calcular a média de um aluno, verificar se ele passou, imprimir a sua ficha... Copiar e colar esse código vezes sem conta torna o programa difícil de ler e ainda mais difícil de corrigir.

Uma **função** (ou **método**) é um bloco de código com nome que executa uma tarefa e pode ser reutilizado sempre que precisarmos. Escrevemos a lógica uma única vez e passamos a chamá-la pelo nome.

Porque usamos métodos

- Reutilização — escreves a lógica uma vez e usas em vários sítios.
- Organização — cada método faz uma coisa, o programa fica mais legível.
- Manutenção — se houver um erro, corriges num único lugar.
- Colaboração — cada elemento da equipa trabalha nos seus métodos.

Função ou método?

Em muitas linguagens fala-se em «funções». Em C#, como tudo vive dentro de classes, o termo correto é **método**. Ao longo deste manual tratamos as duas palavras como sinónimos.

2. Anatomia de um método

Todos os métodos em C# seguem a mesma estrutura. Vejamos um exemplo comentado e os seus componentes:

```
CalcularMedia.cs
1  static double CalcularMedia(double nota1, double nota2)
2  {
3      double media = (nota1 + nota2) / 2;
4      return media;
5  }
```

`static` → pertence à classe (para já, usamos sempre `static`).

`double` → tipo de retorno (o que o método devolve). Se não devolver nada, usa-se `void`.

`CalcularMedia` → nome do método (verbo + substantivo, em PascalCase).

`(nota1, nota2)` → parâmetros: os dados que o método recebe para trabalhar.

`return` → devolve o resultado a quem chamou o método.

Como um método funciona: a máquina de sumo

Uma boa imagem para perceber um método é uma máquina de fazer sumo de laranja. As laranjas que metemos na máquina são os argumentos (os dados que entram). A máquina faz o seu trabalho — espremer — que é a lógica do método. E o que sai, o sumo, é o retorno (o valor devolvido).

Método COM retorno — devolve um valor**Método void — não devolve nada (só faz a ação)**

Os argumentos entram, o método faz o trabalho e o retorno sai. Há métodos void que fazem a ação mas não devolvem sumo nenhum.

Nem todos os métodos devolvem algo: um método **void** é como uma máquina que faz a ação (por exemplo, escrever no ecrã) mas não te entrega sumo. Vamos ver os dois casos a seguir.

2.1 Onde vivem os métodos: a classe e o Main

Em C#, todo o código vive dentro de uma **classe**. Quando o programa arranca, o C# procura um método especial chamado **Main** — é o **ponto de partida**. Tudo o que quisermos executar tem de ser **chamado a partir do Main**.

Repara nos três passos: (1) definimos o método, (2) o Main arranca, (3) chamamos o método lá dentro.

```
Program.cs
1  class Program
2  {
3      // (1) DEFINIÇÃO do método
4      //     Escrito aqui, ainda não corre.
5      static double CalcularMedia(double nota1, double nota2)
6      {
7          return (nota1 + nota2) / 2;
8      }
9
10     // (2) O Main é o ponto de partida do programa
11     static void Main()
12     {
13         // (3) CHAMADA do método – só agora ele corre
14         double resultado = CalcularMedia(14, 16);
15
16         Console.WriteLine(resultado);
17     }
18 }
```

Definir ≠ executar Definir um método não é o mesmo que executá-lo. Um método definido fora do Main, mas nunca chamado no Main, é como uma receita guardada na gaveta: existe, mas ninguém a executa.

3. Tipos de métodos

Ao longo desta secção vamos conhecer os principais tipos de métodos. Cada um traz o seu caso de uso e exemplos completos, prontos a experimentar no Visual Studio Code.

3.1 Métodos void (sem retorno)

A palavra `void` significa «vazio»: o método executa uma ação mas **não devolve nenhum valor** a quem o chamou. Usa-se para fazer coisas — mostrar algo no ecrã, guardar dados, avisar o utilizador.

Quando usar queres executar uma ação (imprimir, guardar, avisar) e não precisas de receber nada de volta.

Exemplo 1 — void com um parâmetro

A ação só acontece quando chamamos o método no Main. E podemos chamá-lo as vezes que quisermos:

```
Program.cs
1  class Program
2  {
3      // Método void: faz uma ação, não devolve valor
4      static void MostrarBoasVindas(string nome)
5      {
6          Console.WriteLine($"Olá {nome}, bem-vindo ao curso!");
7      }
8
9      static void Main()
10     {
11         // Chamamos sem "=", porque não há valor para guardar
12         MostrarBoasVindas("Ana");
13         MostrarBoasVindas("Bruno");
14     }
15 }
```

No ecrã aparece:

```
> Olá Ana, bem-vindo ao curso!
   Olá Bruno, bem-vindo ao curso!
```

Exemplo 2 — void sem parâmetros

Um método void pode nem receber parâmetros. Serve, por exemplo, para imprimir um cabeçalho sempre igual:


```

Program.cs
1  class Program
2  {
3      // void sem parâmetros
4      static void MostrarCabecalho()
5      {
6          Console.WriteLine("=====");
7          Console.WriteLine("    SISTEMA DE GESTÃO DE ALUNOS");
8          Console.WriteLine("=====");
9      }
10
11     static void Main()
12     {
13         MostrarCabecalho();
14     }
15 }

```

Exemplo 3 — void que combina vários dados

Este método recebe o nome e a média e imprime a ficha do aluno em duas linhas:

```

Program.cs
1  class Program
2  {
3      static void MostrarFicha(string nome, double media)
4      {
5          Console.WriteLine($"Aluno: {nome}");
6          Console.WriteLine($"Média: {media}");
7      }
8
9      static void Main()
10     {
11         MostrarFicha("Carla", 15.5);
12     }
13 }

```

No ecrã aparece:

```

> Aluno: Carla
  Média: 15,5

```

Atenção Um método void nunca é usado com «=». Escrever `double x = MostrarFicha(...)`; dá erro, porque não há valor nenhum para guardar.

Exercícios · 3.1 Métodos void (sem retorno)

1. Cria um método void `MostrarCabecalho()` (sem parâmetros) que imprime «==
BIBLIOTECA DA ESCOLA ==».

2. Cria `MostrarLivro(string titulo, int ano)` que imprime «Livro: {titulo} ({ano})».

Dica usa interpolação de strings; repara no tipo `int`.

3. Cria `MostrarDisponibilidade(string titulo, bool disponivel)` que imprime «disponível» ou «emprestado».

Dica decide o texto a partir do `bool`.

4. Cria `MostrarCategoria(char cat)` que imprime «Categoria: {cat}». Chama os quatro métodos no Main.

Dica `char` guarda um único caractere, ex.: 'A'.

3.2 Métodos com retorno

Ao contrário do void, estes métodos calculam um valor e **devolvem-no** a quem os chamou, através de `return`. O tipo declarado (`int`, `double`, `bool`, `string`...) tem de ser igual ao tipo do valor devolvido. Assim que o `return` corre, o método termina de imediato.

Quando usar precisas do resultado de um cálculo ou de uma verificação para usar no resto do programa.

Exemplo 1 — devolver um double (3 formas de o usar)

O valor devolvido não faz nada sozinho: tens de o **aproveitar no Main** — guardá-lo, mostrá-lo, ou usá-lo numa condição.

```

1  class Program
2  {
3      static double CalcularMedia(double n1, double n2)
4      {
5          return (n1 + n2) / 2;
6      }
7
8      static void Main()
9      {
10         // 1) guardar numa variável
11         double media = CalcularMedia(14, 16);
12         Console.WriteLine(media);
13
14         // 2) usar direto dentro do WriteLine
15         Console.WriteLine(CalcularMedia(8, 12));
16
17         // 3) usar numa condição
18         if (CalcularMedia(4, 5) >= 9.5)
19             Console.WriteLine("Aprovado");
20         else
21             Console.WriteLine("Reprovado");
22     }
23 }
```

No ecrã aparece:

```

> 15
10
Reprovado
```

Exemplo 2 — devolver um bool (verdadeiro/falso)

Um método pode devolver um bool. É ideal para uma verificação usada logo a seguir num if:

```
Program.cs
1 class Program
2 {
3     // devolve true se a média chega para passar
4     static bool Aprovado(double media)
5     {
6         return media >= 9.5;
7     }
8
9     static void Main()
10    {
11        double media = 8.0;
12
13        if (Aprovado(media))
14            Console.WriteLine("Passou!");
15        else
16            Console.WriteLine("Não passou.");
17    }
18 }
```

No ecrã aparece:

```
> Não passou.
```

Exemplo 3 — devolver texto (string)

Consoante a média, este método devolve a classificação. O primeiro return que for atingido termina logo o método:

```
Program.cs
1 class Program
2 {
3     static string Classificacao(double media)
4     {
5         if (media >= 16) return "Muito Bom";
6         if (media >= 14) return "Bom";
7         if (media >= 9.5) return "Suficiente";
8         return "Insuficiente";
9     }
10
11    static void Main()
12    {
13        string texto = Classificacao(15);
14        Console.WriteLine(texto);
15    }
16 }
```

No ecrã aparece:

```
> Bom
```

Nota Se chamares `CalcularMedia(14, 16)`; isolado, sem guardar nem usar o resultado, o cálculo é feito mas o valor perde-se. Num método com retorno, isso quase nunca é o que queres.

Exercícios · 3.2 Métodos com retorno

1. Cria `int LivrosRestantes(int total, int emprestados)` que devolve quantos livros ainda estão na estante.
2. Cria `bool TemAtraso(int dias)` que devolve `true` se dias for maior que 0.
Dica a condição `dias > 0` já é um `bool`.
3. Cria `double CalcularMulta(int dias)` que devolve `dias × 0,50 €`.
4. Cria `string Estado(bool disponivel)` que devolve «Disponível» ou «Emprestado». Mostra todos os resultados no Main.

3.3 Parâmetros por valor (o comportamento normal)

Por defeito, quando passamos uma variável a um método, o C# **não entrega a variável original** — entrega uma **cópia** do seu valor. Por isso, alterações feitas dentro do método **não afetam** a variável original lá fora.

```

Program.cs
1  class Program
2  {
3      static void Duplicar(int x)
4      {
5          // altera apenas a CÓPIA local
6          x = x * 2;
7          Console.WriteLine(x);
8      }
9
10     static void Main()
11     {
12         int nota = 8;
13
14         Duplicar(nota);           // envia uma CÓPIA de 8
15         Console.WriteLine(nota); // o original NÃO mudou
16     }
17 }

```

No ecrã aparece:

```

> 16
8

```

Passo a passo

- No Main, a variável nota vale 8.
- Ao chamar Duplicar(nota), o C# copia o 8 para o parâmetro x.
- Dentro do método, x passa a 16 — mas isto é a cópia.
- O método termina e a cópia desaparece.
- De volta ao Main, nota continua a valer 8.

Analogia É como fotocopiar um documento e escrever na fotocópia: o original fica intacto.

Exercícios · 3.3 Parâmetros por valor

1. Cria `void Emprestar(int copias)` que faz copias-- lá dentro. No Main, mostra que a variável original NÃO diminui.

Dica o método recebe uma cópia.

2. Cria `void MudarTitulo(string t)` que faz `t = "ALTERADO"`. Confirma que o título original se mantém.

3. Cria `void AplicarDesconto(double preco)` que faz `preco = preco * 0.9`. Mostra que o preço original fica igual.

4. Num comentário no código, explica por palavras porque é que o valor original não muda.

Dica *pensa na fotocópia.*

3.4 Parâmetro ref (passar a variável original)

Às vezes queremos mesmo que o método altere a variável original. Com a palavra `ref`, o método deixa de receber uma cópia e passa a trabalhar sobre a **própria variável**. Tudo o que mudar lá dentro fica mudado cá fora.

Quando usar precisas de alterar diretamente uma variável que já existe fora do método.

É o mesmo exemplo de cima, mas agora com `ref`. Repara que a palavra `ref` aparece nos dois sítios: na definição e na chamada.

```
Program.cs
1  class Program
2  {
3      static void Duplicar(ref int x)
4      {
5          // altera a PRÓPRIA variável do Main
6          x = x * 2;
7      }
8
9      static void Main()
10     {
11         int nota = 8;
12
13         Duplicar(ref nota);      // repara no "ref" também aqui
14         Console.WriteLine(nota); // o original MUDOU
15     }
16 }
```

No ecrã aparece:

› 16

Regras do ref A variável tem de estar inicializada (com um valor) antes de ser passada com `ref`. Usa `ref` só quando precisas mesmo de alterar o original — na maioria dos casos, o comportamento por valor (3.3) é o mais seguro e o que deves preferir.

Analogia Com `ref` não entregas uma fotocópia — entregas o próprio original para o método escrever nele.

Exercícios · 3.4 Parâmetro ref

1. Reescreve o exercício 1 de 3.3 com `ref` para que as cópias diminuam mesmo no `Main`.
2. Cria `void RegistrarDevolucao(ref int emprestados)` que faz `emprestados--`.

Dica o `ref` aparece na definição e na chamada.

3. Cria `void ReduzirMulta(ref double multa)` que tira 1 € à multa (nunca abaixo de 0).
4. Cria `void MudarCategoria(ref char cat)` que muda a categoria para 'A'. Mostra o valor antes e depois.

3.5 Parâmetro out (devolver mais do que um valor)

O out permite que um método devolva mais do que um resultado ao mesmo tempo. O método é obrigado a atribuir valor ao parâmetro out antes de terminar.

Quando usar um método precisa de devolver vários resultados de uma só vez (ex.: a média E se passou).

```
Program.cs
1  class Program
2  {
3      static double CalcularMedia(double n1, double n2, out bool passou)
4      {
5          double media = (n1 + n2) / 2;
6          passou = media >= 9.5;
7          return media;
8      }
9
10     static void Main()
11     {
12         double m = CalcularMedia(8, 12, out bool situacao);
13         Console.WriteLine($"Média {m} - Passou: {situacao}");
14     }
15 }
```

Exercícios · 3.5 Parâmetro out

1. Cria `void Repartir(int total, int socios, out int porSocio, out int sobram)` que devolve, via out, os livros por sócio e quantos sobram.

Dica usa a divisão / e o resto %.

2. Cria `bool TentarEmprestar(int disponiveis, out string mensagem)` que devolve true/false e uma mensagem via out.

3. Cria `void AnalisarMultas(int dias, out double valor, out bool grave)` com `valor = dias × 0,50` e `grave = dias > 15`.

4. No Main, chama um destes métodos e mostra os dois valores devolvidos.

3.6 Parâmetro params (número variável de argumentos)

Com params podemos passar um número variável de argumentos, que o método recebe como um array.

Quando usar não sabes à partida quantos valores vais receber (ex.: somar N notas).

```
Program.cs
1 class Program
2 {
3     static double Somar(params double[] notas)
4     {
5         double total = 0;
6         foreach (double n in notas)
7             total += n;
8         return total;
9     }
10
11     static void Main()
12     {
13         Console.WriteLine(Somar(10, 12, 14)); // 3 notas
14         Console.WriteLine(Somar(8, 9, 10, 15, 20)); // 5 notas
15     }
16 }
```

Exercícios · 3.6 Parâmetro params

1. Cria `int SomarPaginas(params int[] paginas)` que soma as páginas de vários livros.
2. Cria `double TotalMultas(params double[] multas)` que soma um número variável de multas.
3. Cria `string JuntarAutores(params string[] autores)` que devolve os autores separados por «, ».
Dica *podes usar `string.Join(", ", autores)`.*
4. Chama `SomarPaginas` uma vez com 3 valores e outra com 5.

3.7 Parâmetros opcionais (valor por defeito)

Um parâmetro pode ter um valor por defeito. Se não for passado na chamada, é usado esse valor.

```
Program.cs
1  class Program
2  {
3      static double AplicarBonus(double media, double bonus = 1.0)
4      {
5          return media + bonus;
6      }
7
8      static void Main()
9      {
10         Console.WriteLine(AplicarBonus(12));           // usa bonus = 1.0
11         Console.WriteLine(AplicarBonus(12, 0.5));      // usa bonus = 0.5
12     }
13 }
```

Exercícios · 3.7 Parâmetros opcionais

1. Cria `double CalcularMulta(int dias, double taxa = 0.50)`. Chama com e sem o segundo argumento.
2. Cria `void Emprestar(string titulo, int diasPrazo = 15)` e mostra o prazo usado.
3. Cria `string Etiqueta(string titulo, char categoria = 'G')` que devolve «{titulo} [{categoria}]».
4. Mostra, no ecrã, a diferença entre chamar com e sem o parâmetro opcional.

3.8 Sobrecarga (overloading)

Podemos ter vários métodos com o MESMO nome, desde que os parâmetros sejam diferentes (em número ou tipo). O C# escolhe automaticamente a versão certa.

Quando usar a mesma operação faz sentido com entradas diferentes (duas notas vs. um array de notas).

```

1  class Program
2  {
3      // versão com duas notas
4      static double Media(double a, double b)
5      {
6          return (a + b) / 2;
7      }
8
9      // versão com um array de notas
10     static double Media(double[] notas)
11     {
12         double total = 0;
13         foreach (double n in notas)
14             total += n;
15         return total / notas.Length;
16     }
17
18     static void Main()
19     {
20         Console.WriteLine(Media(10, 20)); // 1ª versão
21         Console.WriteLine(Media(new double[] { 10, 15, 20 })); // 2ª versão
22     }
23 }

```

Exercícios · 3.8 Sobrecarga (overloading)

1. Cria duas versões de `Multa`: `Multa(int dias)` (0,50 €/dia) e `Multa(int dias, double taxa)`.
2. Cria duas versões de `Mostrar`: `Mostrar(string titulo)` e `Mostrar(string titulo, int ano)`.
3. Cria duas versões de `Total`: uma com `(int, int)` e outra com `(double, double)` — mesmo nome, tipos diferentes.

Dica o C# escolhe pela lista de argumentos.

4. Chama as várias versões no `Main` e vê qual é escolhida em cada caso.

4. Arrays

Um array guarda vários valores do mesmo tipo, numa sequência de posições numeradas (índices) que começam em 0. O tamanho é fixo: define-se na criação e não muda.

Criar e aceder

```
Program.cs
1 // criar já com valores
2 double[] notas = { 12, 15, 9, 18 };
3
4 // aceder por índice (começa em 0)
5 Console.WriteLine(notas[0]); // 12
6 Console.WriteLine(notas[3]); // 18
7
8 // alterar um valor
9 notas[2] = 10;
10
11 // número de elementos
12 Console.WriteLine(notas.Length); // 4
```

Percorrer um array

```
Program.cs
1 // com for (temos o índice)
2 for (int i = 0; i < notas.Length; i++)
3     Console.WriteLine($"Nota {i}: {notas[i]}");
4
5 // com foreach (mais simples, sem índice)
6 foreach (double n in notas)
7     Console.WriteLine(n);
```

Passar um array a um método

Um array pode ser recebido como parâmetro. Assim criamos métodos que trabalham sobre qualquer conjunto de notas.

```
Program.cs
1  class Program
2  {
3      static double Media(double[] notas)
4      {
5          double total = 0;
6          foreach (double n in notas)
7              total += n;
8          return total / notas.Length;
9      }
10
11     static void Main()
12     {
13         double[] turmaA = { 10, 14, 18 };
14         Console.WriteLine(Media(turmaA)); // 14
15     }
16 }
```

Exercícios · 4. Arrays

1. Cria um `int[]` com os empréstimos de cada dia da semana e um método que devolve o total.
2. Cria um método que recebe `string[]` de títulos e devolve o título mais comprido.
Dica compara o `.Length` de cada `string`.
3. Cria um método que recebe `double[]` de multas e devolve a maior.
4. Cria um método que recebe `int[]` e conta quantos valores são 0 (livros esgotados).

5. Listas (List<T>)

Uma `List<T>` é como um array, mas de tamanho dinâmico: pode crescer e encolher. O `T` indica o tipo dos elementos (ex.: `List<string>`, `List<double>`). Requer `using System.Collections.Generic;`.

Operações essenciais

```

Program.cs
1  List<string> alunos = new List<string>();
2
3  alunos.Add("Ana");           // adicionar
4  alunos.Add("Bruno");
5  alunos.Add("Carla");
6
7  alunos.Remove("Bruno");      // remover por valor
8  alunos.RemoveAt(0);         // remover por índice
9
10 bool existe = alunos.Contains("Carla"); // true
11 int quantos = alunos.Count;    // número de elementos
12
13 foreach (string nome in alunos)
14     Console.WriteLine(nome);
  
```

Array ou List? Quando usar cada um

Critério	Array	List<T>
Tamanho	Fixo (definido na criação)	Dinâmico (cresce/encolhe)
Adicionar / remover	Não diretamente	Add / Remove / RemoveAt
Contar elementos	.Length	.Count
Quando usar	Nº de elementos fixo	Nº de elementos varia

Regra prática: se não sabes quantos alunos vais ter, usa `List<T>`. Se o número é fixo (ex.: as 3 notas de um período), um array chega.

Exercícios · 5. Listas (List<T>)

1. Cria uma `List<string>` de títulos e um método `Registrar` que adiciona um título e devolve o total.
2. Cria um método que recebe `List<int>` (dias de atraso) e devolve quantos são maiores que 0.

3. Cria um método que recebe `List<string>` e devolve uma nova lista só com os títulos que começam por «O».

Dica usa `titulo.StartsWith("O")`.

4. Cria um método que remove um título de uma `List<string>` se existir.

Dica combina `Contains` com `Remove`.